

RELAZIONE PROGETTO FINALE CYBER

Sara Spadafina

Indice

Indice.....	1
1. Introduzione.....	2
1.1 Architettura simulata.....	2
1.2 Metodologia	2
2. Challenge 1 — Rate Limiter Mancante.....	3
3. Challenge 2 — Operazioni Critiche in GET (CSRF).....	5
4. Challenge 3 — Log Mancanti per Operazioni Critiche	7
5. Challenge 4 — Manomissione Input (SSRF).....	9
6. Challenge 5 — Validazione Contenuto Articolo (Stored XSS)	11
7. Challenge 6 — Uso Non Corretto della Proprietà Fillable (Mass Assignment)	13
8. Conclusioni.....	14
8.1 Possibili sviluppi futuri	14

(Per aggiornare i numeri di pagina: tasto destro sul sommario → Aggiorna campo)

1. Introduzione

Questa relazione documenta il lavoro svolto sul progetto finale “Cyber Blog”, un'applicazione Laravel 11 che simula un'architettura a tre livelli di rete (Internet, DMZ, Rete Interna), con l'obiettivo di individuare e mitigare sei vulnerabilità di sicurezza assegnate, seguendo per ciascuna il ciclo: analisi del codice vulnerabile, esecuzione dell'attacco, implementazione della mitigazione, verifica dell'efficacia tramite riesecuzione dell'attacco.

1.1 Architettura simulata

L'ambiente è stato configurato in locale tramite la modifica del file hosts di sistema, associando tre domini differenti allo stesso indirizzo 127.0.0.1, per simulare provenienze di rete distinte:

- cyber.blog:8000 — applicazione principale, accessibile pubblicamente (DMZ)
- internal.admin:8000 — stessa applicazione, ma accessibile solo per le funzionalità di amministrazione (Rete Interna)
- internal.finance:8001 — componente Financial App, raggiungibile solo dall'applicazione principale (Internal Subnet)

Sono stati verificati il corretto isolamento tra i domini (es. impossibilità di accedere al pannello admin da cyber.blog) e il funzionamento del filtro Referer applicato da Financial App per accettare richieste solo dall'applicazione principale.

1.2 Metodologia

Per ciascuna delle sei challenge è stata seguita la stessa struttura, riportata nei capitoli successivi:

1. Scenario — descrizione della vulnerabilità e del codice responsabile
2. Attacco — dimostrazione pratica dell'exploit, con prova (screenshot, log o risposta HTTP)
3. Mitigazione — descrizione e codice della correzione applicata
4. Verifica — riesecuzione dell'attacco a seguito della correzione, con prova del blocco

2. Challenge 1 — Rate Limiter Mancante

SCENARIO

La rotta pubblica di ricerca articoli non prevedeva alcun meccanismo di limitazione delle richieste, esponendo l'applicazione ad attacchi di tipo Denial of Service (DoS). La rotta era inoltre accessibile senza autenticazione, come esplicitamente previsto dal middleware del controller (except: ['articleSearch']).

```
Route::get('/articles/', [ArticleController::class, 'articleSearch'])
    ->name('articles.search');
// nessun middleware di rate limiting applicato
```

ATTACCO

È stato creato lo script dos-search-attack.sh, che invia 500 richieste HTTP in parallelo (tramite curl in background) verso la rotta di ricerca:

```
URL="http://cyber.blog:8000/articles/?query=test"
REQUESTS=500
for i in $(seq 1 $REQUESTS); do
    curl -s -o /dev/null "$URL" &
done
wait
```

Il server di sviluppo (php artisan serve), gestendo le richieste in modo sequenziale, ha impiegato 90 secondi per smaltire le 500 richieste, evidenziando un evidente rallentamento del servizio durante l'attacco.

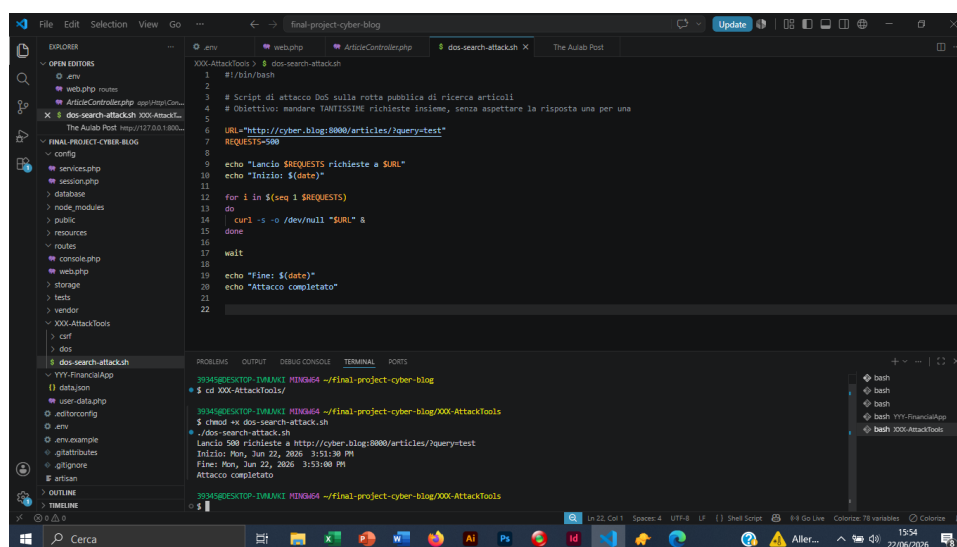


Fig. 1 — Esecuzione dell'attacco DoS senza protezione: 500 richieste smaltite in 90 secondi

MITIGAZIONE

Sono state applicate tre misure complementari:

1. Middleware personalizzato BlockSuspiciousIPs: basato sulla Cache di Laravel, traccia il numero di richieste per indirizzo IP e applica un blocco temporaneo di 1 minuto dopo 5 tentativi in 1 minuto, registrando l'evento nei log applicativi.
2. Protezione differenziata: il middleware BlockSuspiciousIPs è stato applicato solo alla rotta di ricerca (l'operazione più onerosa, segnalata esplicitamente dalla consegna); le rotte di sola navigazione (homepage, lista articoli, candidature) sono state protette con il throttle nativo di Laravel (60 richieste/minuto), più permissivo per non penalizzare l'utilizzo normale.

3. Verifica del rate limiter di Fortify: è stato confermato che il sistema di autenticazione (Laravel Fortify) dispone già di un limite di 5 tentativi di login al minuto per coppia email/IP, giudicato adeguato e lasciato inalterato.

```
protected $maxAttempts = 5;
protected $decayMinutes = 1;
protected $blockMinutes = 1;

public function handle(Request $request, Closure $next): Response
{
    $ip = $request->ip();
    $key = $this->throttleKey($ip);

    if (Cache::has($key . ':blocked')) {
        return response("Troppe richieste. Riprova tra qualche minuto.", 429);
    }
    // ... incremento contatore e blocco dopo $maxAttempts
}
```

Decisione sul rate limiter globale: si è scelto di non applicare un limite uniforme su tutte le rotte, preferendo una protezione proporzionata al costo/rischio di ciascun endpoint (vedi punto 2), unita al fatto che le rotte riservate (writer, revisor, admin) sono già protette dal richiedere autenticazione.

VERIFICA

Ripetendo l'attacco con 15 richieste sequenziali sulla rotta di ricerca, le prime 5 hanno ricevuto risposta 200 (OK); dalla sesta richiesta in poi il server ha risposto con un blocco esplicito, senza eseguire la query di ricerca sul database.

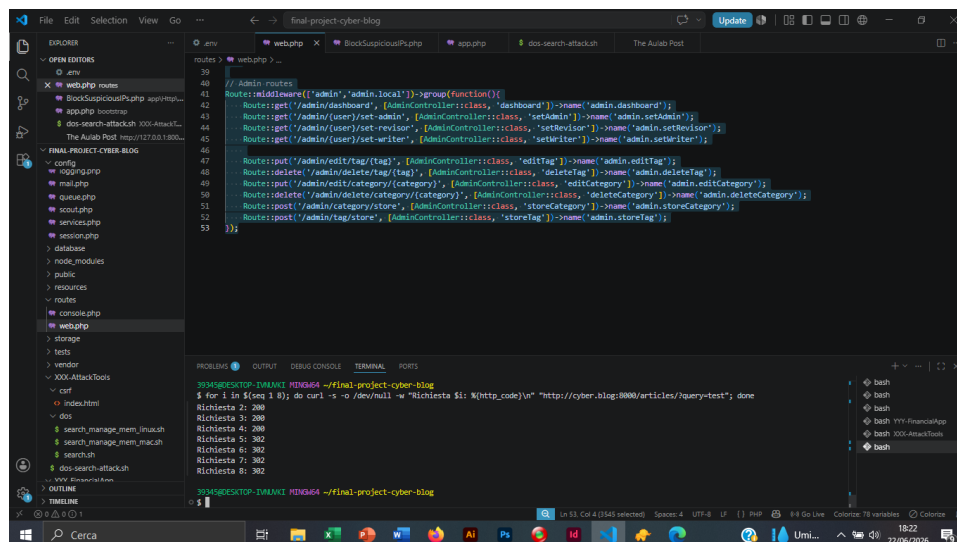


Fig. 2 — Test con 500 richieste dopo la mitigazione: il tempo di gestione si riduce e le richieste eccedenti vengono scartate

Esito: attacco mitigato — blocco confermato dopo 5 richieste/minuto per IP

3. Challenge 2 — Operazioni Critiche in GET (CSRF)

SCENARIO

Le rotte di assegnazione ruolo (set-admin, set-revisor, set-writer), operazioni critiche che modificano i privilegi di un utente, erano implementate come richieste GET, prive della protezione CSRF nativa di Laravel (applicata di default solo alle richieste POST/PUT/PATCH/DELETE generate da form).

```
Route::get('/admin/{user}/set-admin', [AdminController::class, 'setAdmin'])
    ->name('admin.setAdmin');

// nella vista, semplice link cliccabile:
<a href="{route('admin.setAdmin', $user)}" >Enable admin</a>
```

ATTACCO

È stata utilizzata una pagina HTML “trappola” contenente un link nascosto verso la rotta vulnerabile, attivato automaticamente da uno script dopo 5 secondi:

```
<a id="csrf-link" href="http://internal.admin:8000/admin/7/set-admin"
  style="display: none;">Adopt a cute bear for free!</a>
<script>
  setTimeout(() => {
    document.getElementById("csrf-link").click();
  }, 5000);
</script>
```

Con una sessione amministrativa attiva nello stesso browser, la semplice apertura della pagina (camuffata da contenuto innocuo su degli orsetti) ha causato l'assegnazione del ruolo Admin all'utente “attaccante” (Kevin Ross), senza alcuna azione consapevole da parte dell'amministratore. L'evento è stato registrato anche nei log applicativi implementati nella Challenge 3, fornendo una prova indipendente dell'anomalia.

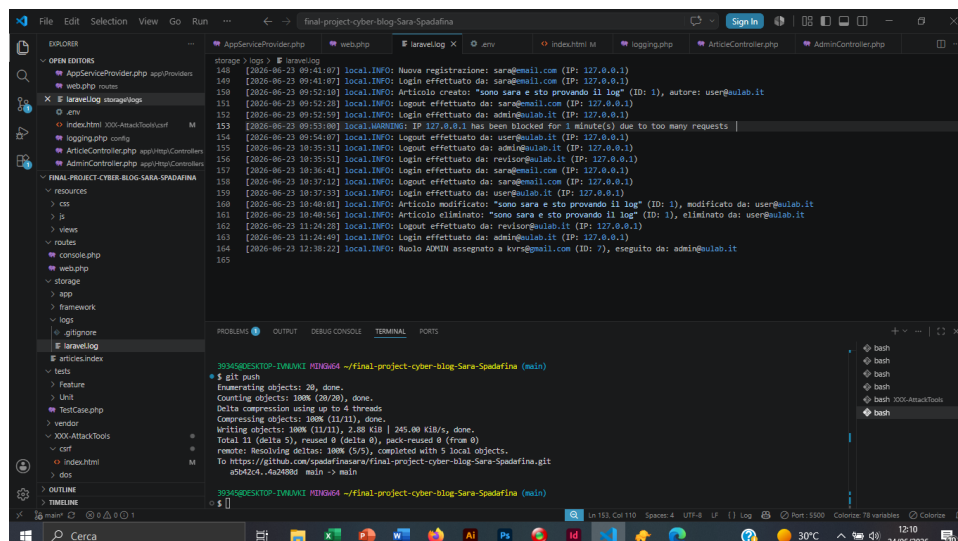


Fig. 3 — Log applicativo: assegnazione del ruolo Admin all'utente attaccante, conseguenza dell'attacco CSRF

MITIGAZIONE

Le tre rotte sono state convertite da GET a PATCH; i link diretti nella vista sono stati sostituiti con form HTML protetti dal token CSRF nativo di Laravel.

```
Route::patch('/admin/{user}/set-admin', [AdminController::class, 'setAdmin'])
    ->name('admin.setAdmin');

<form action="{route('admin.setAdmin', $user)}" method="POST">
  @csrf
```

```
@method('PATCH')  
<button type="submit">Enable admin</button>  
</form>
```

È stata inoltre condotta una verifica sistematica di tutte le rotte del progetto, confermando che nessun'altra operazione di scrittura/modifica fosse esposta tramite GET.

VERIFICA

Ripetendo lo stesso attacco con la pagina trappola, il browser tenta di inviare una richiesta GET (l'unico metodo possibile per un tag <a>) verso una rotta che ora richiede PATCH. Laravel risponde con errore 405 Method Not Allowed, bloccando l'attacco prima che venga eseguita qualsiasi logica applicativa.

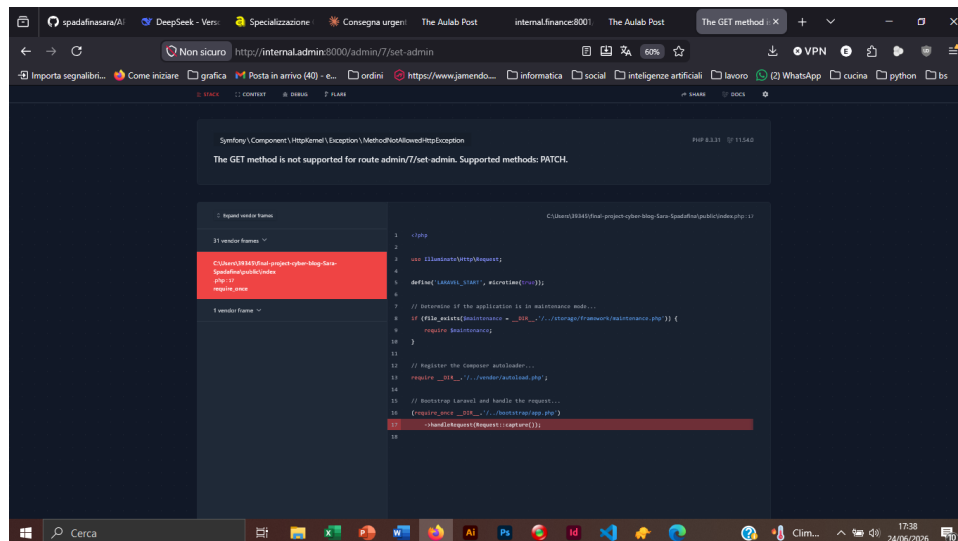


Fig. 4 — La pagina trappola fallisce: errore 405 Method Not Allowed dopo la mitigazione

Esito: attacco mitigato — la richiesta GET viene rifiutata a livello di routing

4. Challenge 3 — Log Mancanti per Operazioni Critiche

SCENARIO

L'applicazione non registrava alcuna informazione relativa a eventi di sicurezza critici (autenticazione, gestione articoli, assegnazione ruoli), violando i principi di accountability e non-repudiation: in caso di incidente, non sarebbe stato possibile ricostruire chi avesse compiuto quale azione.

IMPLEMENTAZIONE

Sono state aggiunte registrazioni di log in quattro punti dell'applicazione:

1. Login, logout, registrazione: tramite i listener degli eventi nativi di Laravel (Login, Logout, Registered), registrati in AppServiceProvider, con email utente e indirizzo IP.
2. Creazione, modifica, eliminazione articolo: in ArticleController (metodi store, update, destroy), con titolo, ID e autore dell'azione.
3. Assegnazione/cambio ruolo: in AdminController, ogni metodo registra l'utente coinvolto, il ruolo assegnato e l'amministratore che ha eseguito l'operazione.
4. Blocco IP per traffico anomalo: il middleware BlockSuspiciousIPs (Challenge 1) registra un log di livello WARNING ad ogni blocco.

```
Event::listen(function (Login $event) {
    Log::info("Login effettuato da: {$event->user->email} (IP: " . request()->ip() . ")");
});

// In ArticleController::destroy
Log::info("Articolo eliminato: \"{$title}\" (ID: $id), eliminato da: " . Auth::user()->email);
```

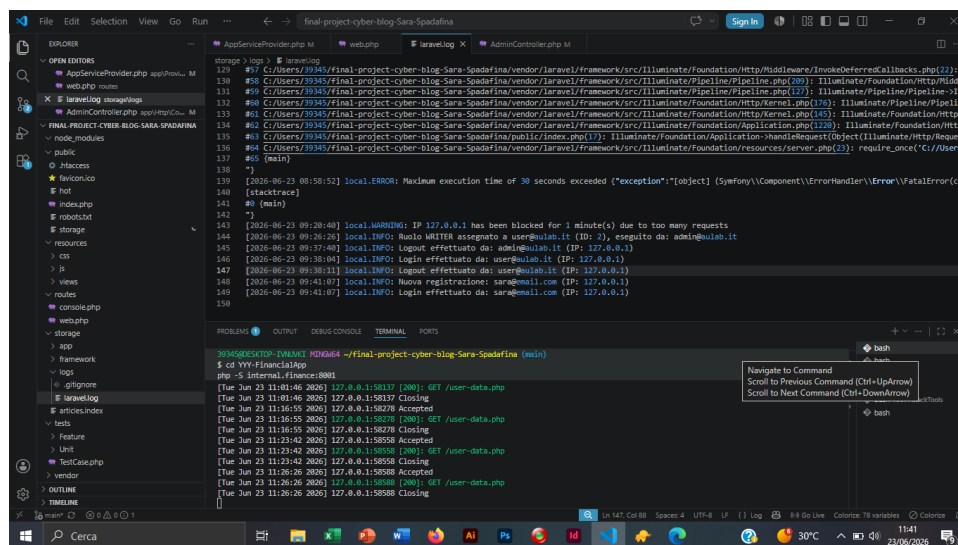


Fig. 5 — Log di login, logout e registrazione, con email utente e indirizzo IP

```

storage > logs > laravel.log
145 [2026-06-23 09:37:40] local.INFO: Logout effettuato da: admin@aulab.it (IP: 127.0.0.1)
146 [2026-06-23 09:38:06] local.INFO: Login effettuato da: user@aulab.it (IP: 127.0.0.1)
147 [2026-06-23 09:38:11] local.INFO: Logout effettuato da: user@aulab.it (IP: 127.0.0.1)
148 [2026-06-23 09:41:07] local.INFO: Nuova registrazione: sara@gmail.com (IP: 127.0.0.1)
149 [2026-06-23 09:41:07] local.INFO: Login effettuato da: sara@gmail.com (IP: 127.0.0.1)
150 [2026-06-23 09:52:18] local.INFO: Articolo creato: "sono sara e sto provando il log" (ID: 1), autore: user@aulab.it
151 [2026-06-23 09:52:28] local.INFO: Logout effettuato da: sara@gmail.com (IP: 127.0.0.1)
152 [2026-06-23 09:52:59] local.INFO: Login effettuato da: admin@aulab.it (IP: 127.0.0.1)
153 [2026-06-23 09:53:00] local.WARNING: IP 127.0.0.1 has been blocked for 1 minute(s) due to too many requests
154 [2026-06-23 09:54:07] local.INFO: Logout effettuato da: user@aulab.it (IP: 127.0.0.1)
155 [2026-06-23 10:35:31] local.INFO: Logout effettuato da: admin@aulab.it (IP: 127.0.0.1)
156 [2026-06-23 10:35:51] local.INFO: Login effettuato da: revisor@aulab.it (IP: 127.0.0.1)
157 [2026-06-23 10:36:41] local.INFO: Login effettuato da: sara@gmail.com (IP: 127.0.0.1)
158 [2026-06-23 10:37:12] local.INFO: Logout effettuato da: sara@gmail.com (IP: 127.0.0.1)
159 [2026-06-23 10:37:23] local.INFO: Login effettuato da: user@aulab.it (IP: 127.0.0.1)
160 [2026-06-23 10:40:01] local.INFO: Articolo modificato: "sono sara e sto provando il log" (ID: 1), modificato da: user@aulab.it
161 [2026-06-23 10:40:56] local.INFO: Articolo eliminato: "sono sara e sto provando il log" (ID: 1), eliminato da: user@aulab.it
162

```

Fig. 6 — Log di creazione, modifica ed eliminazione di un articolo

VERIFICA

Sono stati eseguiti test end-to-end per ciascuna categoria di evento (login, creazione/modifica/eliminazione articolo, assegnazione ruolo), confermando la corretta scrittura nel file storage/logs/laravel.log con timestamp, livello di severità e informazioni identificative.

I log si sono rivelati utili anche come prova indipendente durante la Challenge 2, confermando l'esecuzione anomala di un'azione critica (cambio ruolo) conseguente all'attacco CSRF.

Esito: tracciamento implementato e verificato per tutte le categorie richieste

5. Challenge 4 — Manomissione Input (SSRF)

SCENARIO

La funzionalità di suggerimento notizie (componente Livewire LatestNews), pensata per recuperare articoli da NewsAPI, permetteva di selezionare l'URL di destinazione tramite un menu a tendina HTML facilmente alterabile lato client. Il servizio HttpService, responsabile della chiamata HTTP effettiva, includeva già il dominio interno `internal.finance` nella whitelist dei domini consentiti, senza alcun controllo sul ruolo dell'utente richiedente.

```
// HttpService.php
protected $allowedDomains = ['internal.finance', 'newsapi.org'];
// nessun controllo sul ruolo dell'utente che effettua la richiesta
```

ATTACCO

Tramite gli strumenti di sviluppo del browser, è stata aggiunta una nuova opzione al menu a tendina con valore `http://internal.finance:8001/user-data.php`. Effettuando la richiesta come utente `Writer` (ruolo che secondo le specifiche NON deve poter accedere ai dati finanziari), il server ha recuperato ed esposto nella risposta JSON di Livewire l'intero contenuto del servizio finanziario interno: nominativi, saldi conto, transazioni e dati di carte di credito (numero, scadenza, CVV).

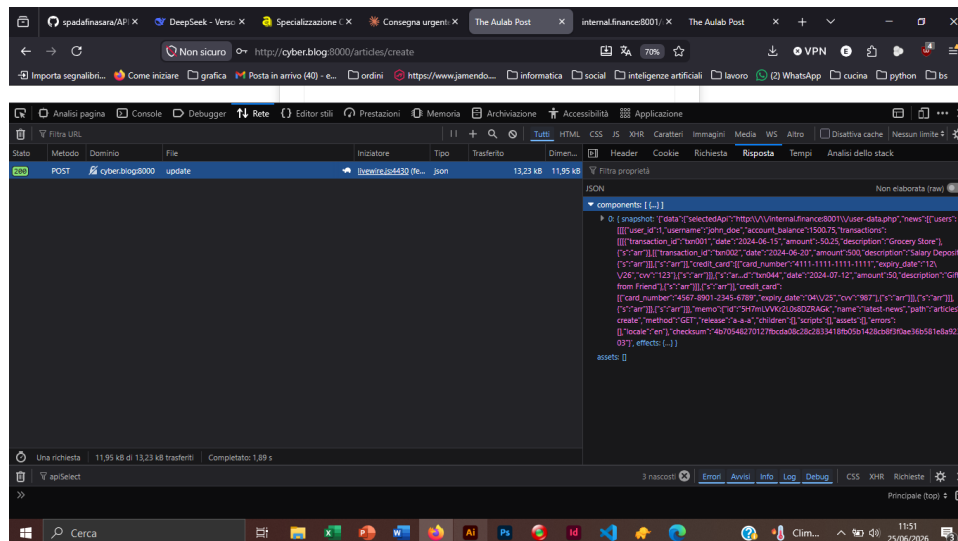


Fig. 7 — Risposta della richiesta Livewire: dati finanziari riservati esposti a un utente `Writer`

MITIGAZIONE

La correzione è stata applicata su due livelli, per garantire una protezione “in profondità”:

```
// LatestNews.php – whitelist degli URL effettivamente consentiti
protected $allowedApis = [
    'https://newsapi.org/v2/top-headlines?country=it&apiKey=...',
    'https://newsapi.org/v2/top-headlines?country=gb&apiKey=...',
    'https://newsapi.org/v2/top-headlines?country=us&apiKey=...',
];

public function fetchNews()
{
    if (!in_array($this->selectedApi, $this->allowedApis)) {
        $this->news = 'Selected source not allowed';
        return;
    }
    // ...
}
```

```
// HttpService.php - controllo del ruolo per il dominio sensibile
if ($parsedUrl['host'] === 'internal.finance') {
    if (!Auth::check() || !Auth::user()->is_admin) {
        return 'Not authorized to access this resource';
    }
}
```

VERIFICA

Ripetendo lo stesso attacco (stessa manomissione del menu a tendina), la richiesta viene rifiutata già a livello del componente Livewire, con messaggio “Selected source not allowed”, senza che la richiesta raggiunga nemmeno il servizio HttpService.

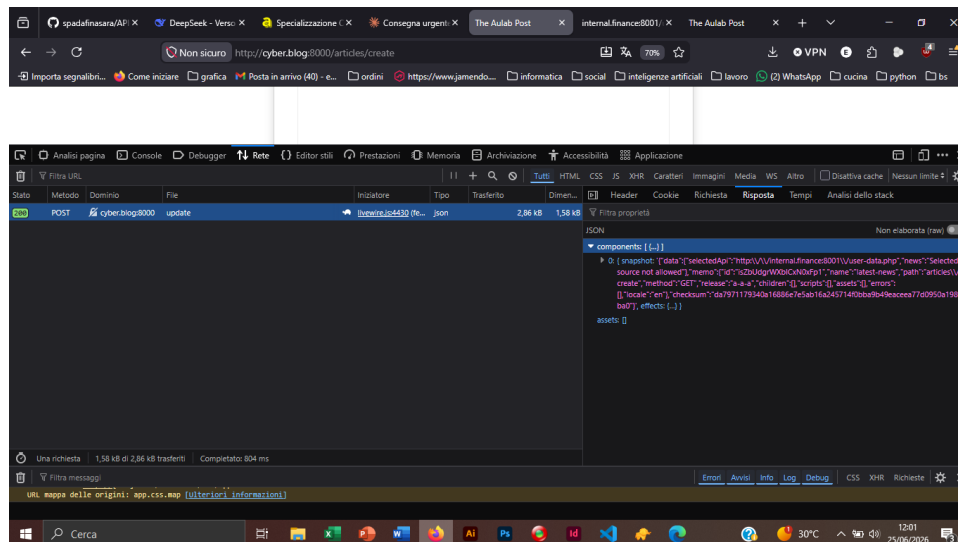


Fig. 8 — La richiesta verso il servizio finanziario viene rifiutata dopo la mitigazione

Esito: attacco mitigato — doppia protezione (whitelist input + controllo ruolo)

6. Challenge 5 — Validazione Contenuto Articolo (Stored XSS)

SCENARIO

Il contenuto degli articoli (campo body), prodotto tramite l'editor TinyMCE, veniva salvato e visualizzato senza alcuna sanificazione lato server. La vista `articles/show.blade.php` utilizzava la sintassi Blade non sicura `{!! !!}`, che stampa l'HTML senza alcun escaping.

```
<p>{!!$article->body!!}</p>
```

ATTACCO

Utilizzando Burp Suite Community Edition in modalità proxy, è stata intercettata la richiesta HTTP di creazione articolo (POST `/articles/store`) dopo la compilazione del form, bypassando così la sanificazione automatica eseguita dall'editor TinyMCE lato client. Il valore del campo body è stato sostituito con il payload seguente prima di inviare la richiesta al server:

```
<script>alert('hacked')</script>
```

Il payload è stato salvato senza alcuna modifica nel database e, alla successiva visualizzazione dell'articolo (anche dalla semplice lista nella dashboard del revisore), lo script si è eseguito automaticamente nel browser della vittima, dimostrando un attacco di tipo Stored XSS.

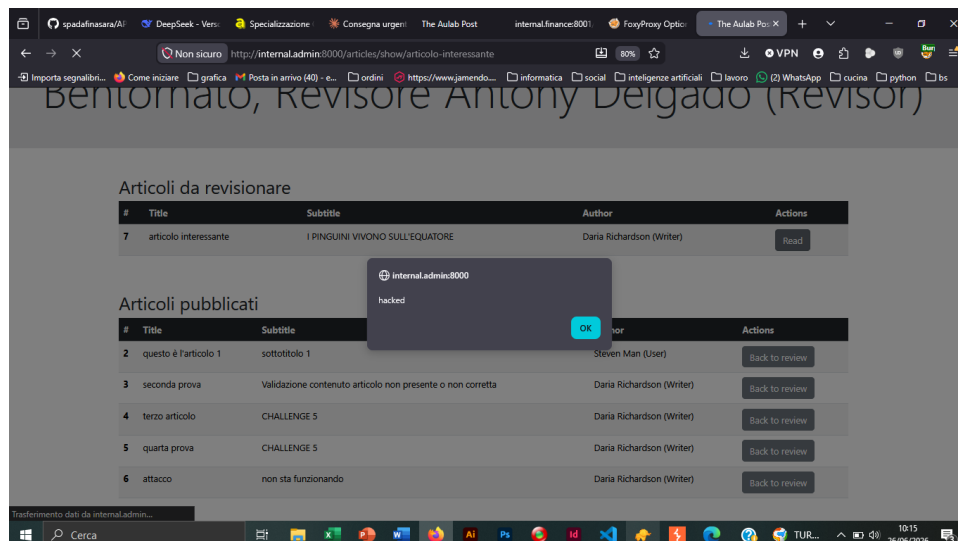


Fig. 9 — Esecuzione automatica dello script malevolo nel browser del revisore

MITIGAZIONE

È stato implementato un metodo `Article::sanitizeBody()` applicato sia in fase di salvataggio (store, update) sia in fase di visualizzazione, per proteggere anche i contenuti già presenti nel database prima della correzione:

```
public static function sanitizeBody($body)
{
    $allowedTags = '<p><b><i><u><strong><em><a><ul><li><ol><br>
                  <h1><h2><h3><h4><blockquote>';

    // Lista bianca: rimuove ogni tag non esplicitamente permesso
    $clean = strip_tags($body, $allowedTags);

    // Rimuove attributi event-handler pericolosi (onerror, onclick, ...)
    $clean = preg_replace('/\son\w+\s*=\s*"([^"])*"/i', '', $clean);
    // Rimuove href/src con scheme javascript:
    $clean = preg_replace('/(href|src)\s*=\s*"([^"])*javascript:[^"]*"'/i, '', $clean);
}
```

```
return $clean;  
}
```

Il metodo viene invocato sia nel controller (al salvataggio) sia nella vista show.blade.php (alla visualizzazione), tramite `Article::sanitizeBody($article->body)`.

VERIFICA

Ripetendo l'attacco con lo stesso payload tramite Burp Suite, il contenuto salvato nel database risulta già privo del tag `<script>`. Inoltre, visualizzando articoli precedentemente compromessi (creati prima della correzione, con payload ancora presenti nel database), il contenuto viene mostrato come testo innocuo, senza alcuna esecuzione di script — confermando la protezione anche retroattiva.

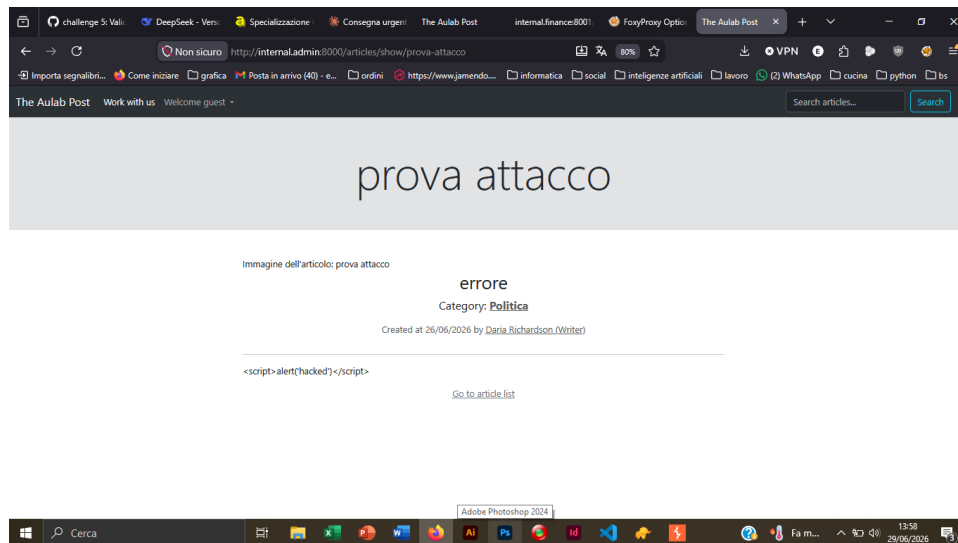


Fig. 10 — Lo stesso payload, salvato prima della correzione, viene mostrato come testo innocuo dopo la mitigazione

Esito: attacco mitigato — protezione a doppio livello (salvataggio e visualizzazione)

7. Challenge 6 — Uso Non Corretto della Proprietà Fillable (Mass Assignment)

SCENARIO

È stata implementata una funzionalità di gestione del profilo utente (modifica nome, email, password), volutamente vulnerabile a scopo didattico. Il modello User presentava i campi `is_admin`, `is_revisor`, `is_writer` all'interno della proprietà `$fillable`, pensata per l'assegnazione di massa (mass assignment) tramite Eloquent; il controller del profilo, inoltre, passava al metodo `update()` l'intero contenuto della richiesta senza alcun filtro.

```
// User.php
protected $fillable = [
    'name', 'email', 'password', 'is_admin', 'is_revisor', 'is_writer'
];

// ProfileController.php
$data = $request->all();
$user->update($data); // nessun filtro sui campi accettati
```

ATTACCO

Tramite gli strumenti di sviluppo del browser, è stato aggiunto manualmente un campo nascosto al form di modifica profilo:

```
<input type="hidden" name="is_admin" value="1">
```

Inviando il form come utente standard (Steven Manson), il campo è stato accettato dal metodo `update()` del modello Eloquent — poiché `is_admin` era presente in `$fillable` — risultando nell'auto-assegnazione del ruolo di amministratore, senza alcuna autorizzazione. La verifica sul database ha confermato il cambio del valore di `is_admin` da 0 a 1.

MITIGAZIONE

I campi relativi ai ruoli sono stati rimossi dalla proprietà `$fillable` del modello User. È stato verificato che la rimozione non comportasse effetti collaterali: le funzionalità di assegnazione ruolo in `AdminController` operano tramite assegnazione diretta dell'attributo e salvataggio esplicito, un meccanismo non soggetto alla protezione fillable.

```
protected $fillable = [
    'name', 'email', 'password',
];

// AdminController.php — invariato, non passa da $fillable
$user->is_admin = true;
$user->save();
```

VERIFICA

Ripetendo lo stesso attacco (stesso campo nascosto `is_admin=1` nel form di profilo), il campo viene scartato silenziosamente da Eloquent. Il salvataggio degli altri campi (nome, email, password) continua a funzionare correttamente, mentre il valore di `is_admin` nel database resta inalterato (0), confermato tramite verifica diretta sul database.

Esito: attacco mitigato — i campi sensibili non sono più assegnabili in massa

8. Conclusioni

Le sei challenge assegnate sono state completate seguendo un approccio sistematico: per ciascuna vulnerabilità è stato dimostrato un attacco funzionante, è stata implementata e motivata una mitigazione, e l'attacco è stato ripetuto per verificarne il blocco effettivo.

Challenge	Vulnerabilità	Stato
1	Rate limiter mancante	Completata
2	Operazioni critiche in GET (CSRF)	Completata
3	Log mancanti per operazioni critiche	Completata
4	Manomissione input (SSRF)	Completata
5	Validazione contenuto articolo (Stored XSS)	Completata
6	Uso non corretto di fillable (Mass Assignment)	Completata

8.1 Possibili sviluppi futuri

Tra le attività bonus previste dalla consegna, le seguenti rappresentano naturali estensioni del lavoro svolto:

- Rate limiter sul login Fortify — già verificato presente e adeguato (5 tentativi/minuto)
- Pagina di clickjacking, da adattare al progetto corrente
- Verifica della funzione di ricerca full-text con Laravel Scout
- Scansione con OWASP ZAP per individuare ulteriori margini di miglioramento
- Adozione delle Policy di Laravel per centralizzare la logica di autorizzazione

Il lavoro svolto ha permesso di consolidare una metodologia pratica per l'identificazione e la correzione di vulnerabilità web comuni (OWASP Top 10), con particolare attenzione alla verifica end-to-end di ogni correzione tramite riesecuzione dell'attacco originale.